01-data-plane

Data Plane — WireGuard core, pluggable transports, obfuscation, routing

Revision: 1 **Last modified:** 2026-06-25T00:00:00Z

> Master technical specification — document 01 of the HelixVPN set. Scope: the **Rust data plane** shared byte-for-byte by client, connector, and gateway edge. This is a SPEC (describe the implementation; do not build the product). Source evidence cited inline by id, e.g. [04_ARCH §3], [04_P0 §4.2], [11_MST], [SYNTHESIS].

---

0. Position in the system & what this document owns

HelixVPN is a self-hostable overlay network with a privacy-VPN front end: three roles — **Connector** (network-side outbound-only agent advertising CIDRs) ⇄ **Gateway** (public VPS: control + data plane) ⇄ **Client** (Mullvad-style end-user app) [04_ARCH §1.1, 05_YBO, SYNTHESIS §1]. This document specifies **only the data plane** — the code that moves already-encrypted bytes — which is the same Rust workspace on all three roles [04_ARCH §3.2, §5.5]. It does **not** specify the Go control plane (doc 02), the `WatchNetworkMap` protobuf (doc 03), client UI (doc 05), or platform tunnel shims beyond the TUN handoff contract (doc 06).

The single most important architectural constraint that governs every line below [04_ARCH §0, §2.2]: *WireGuard is the cryptographic core everywhere; obfuscation/transport is a swappable layer **under***

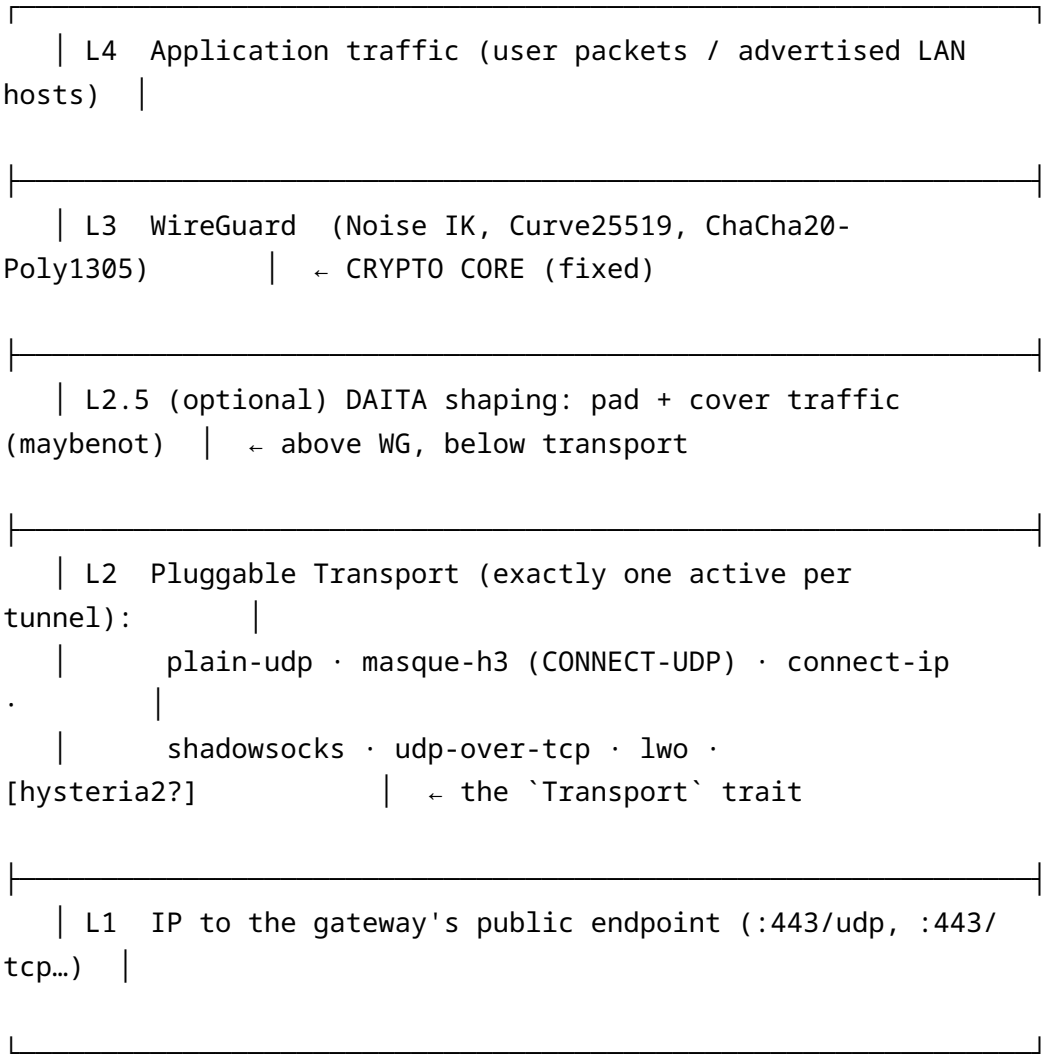
WireGuard, never a fork of WG crypto. Mullvad's "QUIC mode" is not a separate protocol — it is WireGuard tunnelled over MASQUE/HTTP-3 [04_ARCH §0, research synthesis D1]. HelixVPN keeps WG as the L3 crypto and treats QUIC/Shadowsocks/ UDP-over-TCP/LWO as interchangeable L2 carriers of opaque WG datagrams.

0.1 Non-negotiable data-plane invariants

#	Invariant	Source
I1	The transport layer **never sees plaintext**; it carries only already-encrypted WG datagrams.	[04_P0 §4.2]
I2	The transport carries **unreliable datagrams**, never an ordered byte stream — preserving WG's own loss semantics, avoiding head-of-line blocking.	[04_P0 §4.2 design note]
I3	Control plane is **never in the packet path**; if control is down, existing tunnels keep forwarding (fail-static).	[04_ARCH §2.1]
I4	**One transport crate, three consumers** (client, connector, edge): the code that obfuscates on the client is the same code that de-obfuscates on the edge.	[04_ARCH §3.2, §5.5]
I5	**No-logging by construction** in the data plane: only	[04_ARCH §2.7, §7]

#	Invariant	Source
	aggregate counters, never per-connection/per-packet durable state.	
I6	Default-**deny**: no peer reaches anything without an explicit compiled policy rule expressed as `AllowedIPs` + an edge verdict map.	[04_ARCH §3.4, §7]

1. Layering model



WireGuard owns confidentiality, integrity, and roaming. The transport layer **only changes how encrypted WG datagrams look on the wire** so DPI cannot fingerprint or block them [04_ARCH §3.1]. DAITA is orthogonal to *which* transport runs (works with plain-udp or masque alike) [04_P2 §2.2] and is specified at §9.

---

2. Crate layout (the data-plane workspace)

The Phase-0 interfaces are designed to **survive into Phase 1** [04_P0 §4]. The workspace is a decoupled, reusable Rust component per constitution §11.4.28/.74 (its own `vasic-digital` repo, snake_case, flat submodule) [SYNTHESIS §9].

```
helix-core/                          # Cargo workspace; repo:
vasic-digital/helix_core
├── Cargo.toml                       # [workspace]
├── crates/
│   ├── helix-transport/             # the pluggable L2 carrier —
shared client+edge+connector
│   │   ├── src/lib.rs               #   Transport trait +
TransportConfig + dial() + registry
│   │   ├── src/plain_udp.rs         #   §3.2
│   │   ├── src/masque.rs            #   §3.3  (quinn + h3 +
CONNECT-UDP/HTTP-Datagram)
│   │   ├── src/connect_ip.rs        #   §3.4  (RFC 9484, advanced
— no inner WG)
│   │   ├── src/shadowsocks.rs       #   §3.5
│   │   ├── src/udp_over_tcp.rs      #   §3.6
│   │   ├── src/lwo.rs               #   §3.7
│   │   ├── src/hysteria2.rs         #   §3.8  (option, decision
D1)
│   │   └── src/error.rs             #   TransportError taxonomy
│   ├── helix-wg/                    # §4  boringtun wrapper:
handshake, encrypt/decrypt, timers
│   ├── helix-tun/                   # OS TUN abstraction (Linux
native; shims inject the fd)
│   ├── helix-daita/                 # §9  maybenot shaping stage
│   ├── helix-route/                 # §6–§8  overlay addressing,
4via6, ACL→verdict-map compiler
│   ├── helix-core/                  # §5  orchestrator: ties tun ⇄
wg ⇄ daita ⇄ transport; status
│   └── helix-ffi/                   # flutter_rust_bridge surface
(doc 05)
```

```
└── bin/
    ├── helix-client.rs             # Linux CLI client
    ├── helix-connector.rs          # Linux CLI connector
(advertise/route mode)
    └── helix-edge.rs               # gateway data-plane edge
(MASQUE termination) — see D5
```

The same `helix-edge` binary on the gateway is the third consumer of `helix-transport` [04_ARCH §11, SYNTHESIS §6]. Decision D5 (Rust vs Go edge) is settled by the Phase-0 G4 benchmark (§11); this spec assumes the **Rust** edge for the single-implementation guarantee and notes the Go fallback at §11.

---

3. The Transport trait — the single most important interface

Everything obfuscation-related hides behind one trait. `plain_udp`, `masque`, … implement it; the client, connector, and edge all consume it. **One trait, three consumers, N transports** [04_P0 §4.2].

3.1 Trait, config, error, and the `dial()` ladder

```rust
// helix-transport/src/lib.rs
use async_trait::async_trait;
use bytes::Bytes;
use std::net::SocketAddr;

/// A bidirectional carrier for ALREADY-ENCRYPTED WireGuard
    datagrams.
/// The transport NEVER sees plaintext (I1); it carries unreliable
    datagrams,
/// never an ordered stream (I2). Implementations are cancel-safe.
#[async_trait]
pub trait Transport: Send + Sync {
    /// Send exactly one WG datagram toward the peer endpoint.
    async fn send(&self, datagram: Bytes) -> Result<(),
        TransportError>;

    /// Receive the next WG datagram from the peer (cancel-safe;
        usable in `select!`).
    async fn recv(&self) -> Result<Bytes, TransportError>;

    /// Stable label for logs/metrics: "plain-udp", "masque-h3",
        "shadowsocks", …
```

```rust
    fn kind(&self) -> &'static str;

    /// MTU the upper layer (WG) may use over this transport,
        AFTER per-transport overhead (§10).
    fn effective_mtu(&self) -> u16;

    /// Snapshot of liveness (RTT EWMA, last-recv age) the
        orchestrator uses for ladder decisions.
    fn health(&self) -> TransportHealth;

    /// Graceful close (flush QUIC, FIN the TCP carrier).
        Idempotent.
    async fn close(&self) -> Result<(), TransportError>;
}

#[derive(Clone, Debug)]
pub struct TransportHealth {
    pub rtt_ewma_ms: Option<u32>,
    pub last_recv_age_ms: u64,
    pub send_errors: u32,
}

/// One variant per L2 carrier. The escalation ladder constructs
        the next
/// variant on repeated handshake failure (§5.3). All variants
        resolve their
/// concrete endpoint + secrets from the NetworkMap pushed by the
        coordinator (doc 03).
#[derive(Clone, Debug)]
pub enum TransportConfig {
    PlainUdp     { peer: SocketAddr, bind: SocketAddr },
    MasqueH3     { url: String, sni: String, bind: SocketAddr,
        congestion: Congestion },
    ConnectIp    { url: String, sni: String, bind:
        SocketAddr }, // RFC 9484, advanced (§3.4)
    Shadowsocks  { peer: SocketAddr, method: SsMethod, psk:
        SecretBytes },
    UdpOverTcp   { peer: SocketAddr, tls_sni: Option<String> },
    Lwo          { peer: SocketAddr, bind: SocketAddr,
        session_key: SecretBytes },
    #[cfg(feature = "hysteria2")]
    Hysteria2    { url: String, sni: String, salamander_pw:
        SecretBytes }, // decision D1 (§3.8)
}

#[derive(Clone, Copy, Debug)] pub enum Congestion { Cubic, Bbr }
#[derive(Clone, Copy, Debug)] pub enum SsMethod  {
        Chacha20Poly1305, Aes256Gcm }
```

```rust
/// Build a live transport from config. Returns within a bounded
        dial timeout;
/// on `DialTimeout`/`HandshakeFailed` the orchestrator escalates
        to the next rung (§5.3).
pub async fn dial(cfg: TransportConfig) -> Result<Box<dyn
        Transport>, TransportError> { /* … */ }

// helix-transport/src/error.rs
#[derive(thiserror::Error, Debug)]
pub enum TransportError {
    #[error("dial timed out")]            DialTimeout,
    #[error("handshake failed: {0}")]
        HandshakeFailed(String),  // → triggers ladder escalation
    #[error("peer endpoint blocked")]
        EndpointBlocked,          // DPI/firewall verdict
    #[error("transport closed")]          Closed,
    #[error("oversize datagram {0} > mtu")] Oversize(usize),
    #[error("io: {0}")]                   Io(#[from]
        std::io::Error),
    #[error("quic: {0}")]                 Quic(String),
}
```

Design rationale (I2): because WG datagrams are independent and loss-tolerant, the transport carries unreliable datagrams. For MASQUE this means **QUIC DATAGRAM frames (RFC 9221)** carried as **HTTP Datagrams (RFC 9297)**, *not* a QUIC stream — preserving WG's loss semantics and avoiding head-of-line blocking [04_P0 §4.2, §5.1]. This is the half of S3 that earns Phase 0.

3.2 `plain-udp` — the baseline (Phase 0 / always available)

The default, lowest-latency, lowest-CPU path on unrestricted networks [04_ARCH §3.2]. It also sets the throughput baseline every other transport is measured against (≥80% of bare link is the G1 gate) [04_P0 §0, §8].

```rust
// helix-transport/src/plain_udp.rs
pub struct PlainUdp { sock: tokio::net::UdpSocket, peer:
        SocketAddr, health: HealthCell }

#[async_trait]
impl Transport for PlainUdp {
    async fn send(&self, dg: Bytes) -> Result<(), TransportError>
        {
        self.sock.send_to(&dg, self.peer).await?; Ok(())
    }
    async fn recv(&self) -> Result<Bytes, TransportError> {
        let mut buf = vec![0u8; 1500];
```

```
        let n = self.sock.recv(&mut buf).await?;
        self.health.mark_recv();
        buf.truncate(n); Ok(Bytes::from(buf))
    }
    fn kind(&self) -> &'static str { "plain-udp" }
    fn effective_mtu(&self) -> u16 { 1420 }      // standard WG-
        over-IPv4 (§10)
    fn health(&self) -> TransportHealth { self.health.snapshot() }
    async fn close(&self) -> Result<(), TransportError> { Ok(()) }
}
```

3.3 `masque-h3` — CONNECT-UDP over HTTP/3 (the headline obfuscation)

The "QUIC mode" — Mullvad's actual mechanism, WG-over-MASQUE [04_ARCH §0, §3.3]. The client wraps each WG datagram in an HTTP/3 **CONNECT-UDP** flow (RFC 9298) to the gateway's `:443/udp` HTTP/3 listener; to a passive observer the flow is indistinguishable from a browser doing HTTP/3 [04_ARCH §3.3, 04_P0 §5].

```
client WG datagram (Bytes)
  └▶ HTTP Datagram  (RFC 9297, context-id = 0 for the proxied UDP
payload)
       └▶ QUIC DATAGRAM frame (RFC 9221, unreliable — matches WG
semantics, I2)
            └▶ QUIC / HTTP-3 connection to https://gateway:443
(looks like web)
                 └▶ EDGE: extract HTTP Datagram → WG datagram →
kernel WG fast path
```

RFC stack: **RFC 9298** (CONNECT-UDP / Proxying UDP in HTTP), **RFC 9297** (HTTP Datagrams & Capsule Protocol), **RFC 9221** (unreliable QUIC datagrams) [04_P0 §5.1, research-masque]. The CONNECT-UDP request establishes the proxied UDP "flow" once at `dial()`; thereafter WG datagrams ride as HTTP Datagrams with **no per-packet HTTP round trip** [04_P0 §5.1].

```
// helix-transport/src/masque.rs  (the parts that matter)
pub struct MasqueTransport {
    conn: quinn::Connection,          // established QUIC/H3
        connection to gw:443
    flow_ctx: u64,                    // CONNECT-UDP context-id
        (0) established at dial()
    health: HealthCell,
}
```

```rust
#[async_trait]
impl Transport for MasqueTransport {
    async fn send(&self, wg: Bytes) -> Result<(), TransportError>
        {
        let http_dg = encode_http_datagram(self.flow_ctx,
        &wg); // RFC 9297 framing
        self.conn.send_datagram(http_dg).map_err(|e|
        TransportError::Quic(e.to_string()))?;

        Ok(())                                                  //
        RFC 9221 QUIC datagram
    }
    async fn recv(&self) -> Result<Bytes, TransportError> {
        let dg = self.conn.read_datagram().await.map_err(|e|
        TransportError::Quic(e.to_string()))?;
        self.health.mark_recv();

        Ok(decode_http_datagram(dg)?)                           //
        strip framing → WG bytes
    }
    fn kind(&self) -> &'static str { "masque-h3" }
    fn effective_mtu(&self) -> u16 { 1280 }   // QUIC overhead
        eats headroom — measure & tune (§10)
    fn health(&self) -> TransportHealth { self.health.snapshot() }
    async fn close(&self) -> Result<(), TransportError> {
        self.conn.close(0u32.into(), b"bye"); Ok(()) }
}
```

Building blocks & maturity caveat [04_P0 §5.2]: `quinn` (mature QUIC, exposes `send_datagram`/`read_datagram`); `h3` (HTTP/3 on quinn, less battle-tested than Go's stack); **MASQUE CONNECT-UDP handling is thin-to-absent as a turnkey Rust crate** — expect to implement CONNECT-UDP request + HTTP-Datagram framing by hand on top of `h3` + `quinn` datagrams. This relative immaturity vs Go's `masque-go` is itself the input to decision D5 (edge language, §11).

Masquerade [04_ARCH §3.3, 04_P0 §5.4]: the edge `:443` listener serves a believable decoy site to anything that is *not* a valid CONNECT-UDP flow (probes/scanners) — native edge behavior replacing the original doc's Nginx-camouflage. Verify the flow classifies as HTTP/3 with no WG signature via `tshark` (the G2 wire-fingerprint check) [04_P0 §8].

Loss resilience is the reason mobile got this feature: under `netem loss 5%`, masque/QUIC must sustain higher goodput than the UDP-over-TCP strawman [04_P0 §5.3]. Congestion control is selectable (`Cubic` default; `Bbr` for lossy mobile — the Hysteria "Brutal" lineage tuning knob is a useful reference for `quinn` window ratios) [04_ARCH §14].

3.4 `connect-ip` — IP-over-HTTP/3 (RFC 9484, advanced)

An alternative datapath that proxies **IP packets** (not inner WG) directly over HTTP/3 via **CONNECT-IP** [04_ARCH §3.2, research-masque]. Use when a native IP-over-H3 path is desired without the inner WG layer (e.g. the gateway acts as a true IP router for the flow). **Spec stance:** ship behind a feature flag in Phase 2+; the default and recommended path remains inner-WG-over-`masque-h3` (§3.3) so the crypto core (WG) and the no-logging/ACL model are unchanged. CONNECT-IP loses I1's "transport never sees plaintext" property at the gateway (the gateway terminates IP), so it is **not** offered to privacy clients by default — only for explicit site-to-site routing where the gateway is already a trusted router. Recommendation: implement after `masque-h3` is proven; do not block the MVP on it.

3.5 `shadowsocks` — WG-in-Shadowsocks (China-grade DPI)

WG datagrams wrapped in a Shadowsocks AEAD stream — the canonical "looks like random/TLS-ish TCP" evasion where even QUIC is throttled [04_ARCH §3.2, 04_P2 §1.1].

```
// helix-transport/src/shadowsocks.rs  (sketch)
pub struct ShadowsocksTransport {
    stream: shadowsocks_crypto::AeadStream,   // chacha20-poly1305 / aes-256-gcm
    // session keys derived from a pre-shared transport password, SEPARATE from WG keys
}
// send: frame WG datagram with 2-byte length prefix → AEAD encrypt → TCP
// recv: AEAD decrypt → deframe → WG datagram
```

Reuse `shadowsocks-rust` crypto primitives rather than re-implementing AEAD framing [04_P2 §1.1]. Because it rides TCP it carries the head-of-line-blocking caveat; the ladder prefers it only when UDP/QUIC are blocked. `effective_mtu()` reports a conservative value to leave room for the 2-byte length prefix + AEAD tag (§10).

3.6 `udp-over-tcp` — last resort when all UDP is blocked

WG datagrams length-prefixed over a single TCP connection (matches Mullvad's `udp2tcp`) [04_ARCH §3.2, 04_P2 §1.2]. Accept the head-of-line-blocking penalty; it exists purely to keep a tunnel *possible* on the most hostile networks. Optional TLS wrap (`tls_sni`) to look like HTTPS. Lowest rung of the ladder.

3.7 `lwo` — lightweight WG obfuscation (cheap evasion)

Per-session keyed obfuscation of the WG header bytes that DPI signatures key on (message-type / reserved fields) plus randomized padding — near-zero-cost evasion of *naive* WG fingerprinting without QUIC overhead [04_ARCH §3.2, 04_P2 §1.3]. Phase 1 ships a basic XOR/padding scheme; Phase 2 hardens it to a proper per-session keyed scheme. Mullvad's "LWO" is the design reference. It is the **first** escalation rung above plain-udp because it is the cheapest non-default option.

3.8 `hysteria2` — decision D1 (option, not default)

> **DECISION D1 — primary obfuscating transport. SURFACED, not silently resolved** [SYNTHESIS §3 D1, 04_ARCH §0, 11_MST].
>
> - **Camp A (recommended, CLD/04_ARCH):** `masque-h3` is the primary obfuscating transport. It is *true Mullvad parity* (Mullvad's QUIC mode IS WG-over-MASQUE) and a **single Rust implementation** shared client↔edge (I4). WG stays the crypto core unchanged.
> - **Camp B (plurality of the 10-LLM analyses): Hysteria2 + Salamander** (QUIC + obfs, turnkey, BBR/Brutal congestion) as the primary obfuscating transport with WG as fallback — *ships faster* because Hysteria2 is a mature, turnkey QUIC-obfs stack.
>
> **Recommendation:** adopt **Camp A (`masque-h3`) as the primary** for MVP, because (1) it preserves the "WG crypto core, pluggable transport" invariant (a Hysteria2-primary design makes Hysteria2 *the* protocol, not a carrier under WG, weakening I1/I4); (2) it is the only path that is literal Mullvad parity; (3) it keeps one Rust transport crate. **Provide `hysteria2` as an additional `Transport` impl behind a Cargo feature** so the turnkey QUIC-obfs path is available as a ladder rung and as a Phase-0 unblock if `masque-h3`-in-Rust proves painful (the documented G2 fallback: "prototype MASQUE first in Go to unblock G2, then port" [04_P0 §13]). Asymmetric-per-leg (decision D6, §11) can additionally place Hysteria2 on the user↔gateway leg only [11_MST].

```
// helix-transport/src/hysteria2.rs  — feature = "hysteria2"
#[cfg(feature = "hysteria2")]
pub struct Hysteria2Transport { conn: quinn::Connection, /*
        salamander obfs state */ health: HealthCell }
```

```
// Carries WG datagrams over Hysteria2's QUIC datagram channel +
        Salamander packet obfuscation.
// Reuses the SAME quinn dependency as masque-h3; the obfs
        differs, the WG-datagram contract is identical (I2).
```

---

4. `helix-wg` — the boringtun wrapper (the WG crypto core handle)

boringtun is the right choice: pure-Rust userspace WireGuard, no kernel module, runs identically on Linux/Android/iOS — exactly the iOS path (no kernel WG on iOS regardless) [04_P0 §4.4]. On Linux the kernel WG fast path is used where available, `boringtun` userspace as fallback (decision: ship both, default to kernel) [04_ARCH §13]. The wrapper owns WG's `Tunn` state machine, pumps its timer, and routes its four output verdicts.

```rust
// helix-wg/src/lib.rs  (sketch)
pub struct WgPeer { tunn: boringtun::noise::Tunn, /* keys,
        endpoint, allowed_ips */ }


pub enum WgVerdict<'a> {
    WriteToTransport(&'a [u8]),  // encrypted datagram → hand to
        the Transport (§3)
    WriteToTun(&'a [u8]),        // decrypted IP packet → hand to
        the TUN
    Nothing,
    Err(WgError),
}


impl WgPeer {
    /// Inbound: bytes arrived from the transport → maybe
        plaintext for the TUN,
    /// or a handshake reply to send back out the transport.
    pub fn handle_transport_in(&mut self, datagram: &[u8],
        scratch: &mut [u8]) -> WgVerdict { /* … */ }
    /// Outbound: a plaintext IP packet from the TUN → encrypted
        datagram for transport.
    pub fn handle_tun_out(&mut self, ip_pkt: &[u8], scratch: &mut
        [u8]) -> WgVerdict { /* … */ }
    /// Call ~every 100-250 ms: emits keepalives / rekeys.
    pub fn tick(&mut self, scratch: &mut [u8]) -> WgVerdict
        { /* … */ }
}
```

Crypto parameters are fixed by WireGuard and never altered [04_ARCH §2.2, SYNTHESIS §2]: Noise IK handshake, Curve25519 ECDH, ChaCha20-Poly1305 AEAD, BLAKE2s. Device private keys are generated on-device and **never leave the device** (doc 02 enrollment) [04_ARCH §7, SYNTHESIS §7]. Post-quantum is an additive **PSK** layer (ML-KEM/FIPS-203, hybrid-never-PQ-only) in Phase 2 — it slots into `Tunn`'s preshared-key field with no transport change [04_P2, SYNTHESIS §4]; specified in the security doc, noted here only as a non-breaking WG-layer seam.

---

5. Orchestrator, status stream, and auto transport selection

5.1 The three loops + the connector posture

The orchestrator (`crates/helix-core`) runs three loops [04_P0 §4.5]: `tun → wg → (daita) → transport`, `transport → wg → tun`, and `timer tick`. The **connector** is the *same core* with a different routing posture: instead of capturing the device's default route it (a) advertises its served CIDRs (e.g. `10.10.0.0/24`) in its map, and (b) NATs/forwards decapsulated packets into the LAN interface and back. One binary, a `--mode={client|connector}` flag [04_P0 §4.5].

```
client:    [app traffic] → TUN → WG encrypt → [DAITA] → Transport ──► gateway
connector: gateway ──► Transport → WG decrypt → forward into served CIDR (and reverse)
```

5.2 The status enum (the real-time UX seam)

The orchestrator exposes an event stream via **`tokio::sync::broadcast`** — the same enum the FFI surfaces to Flutter (doc 05) and what proves the real-time status UX [04_P0 §4.5, §9].

```rust
// helix-core/src/status.rs  (mirrored across the FFI boundary, doc 05)
#[derive(Clone, Debug)]
pub enum TunnelStatus {
    Connecting,
    Handshaking,
    Connected { transport: String, rtt_ms: u32 },  // e.g. "masque-h3", 23
    Reconnecting,
```

```
    Down { reason: String },
}

pub struct Orchestrator {
    status_tx: tokio::sync::broadcast::Sender<TunnelStatus>,  //
        fan-out to FFI + metrics + ladder
    // … wg peers, active transport, daita stage, reconciler
        handle …
}
impl Orchestrator {
    pub fn subscribe(&self) ->
        tokio::sync::broadcast::Receiver<TunnelStatus> {
        self.status_tx.subscribe() }
}
```

5.3 Auto transport selection — the escalation ladder

Selection is **automatic with manual override** — Mullvad's exact UX [04_ARCH §3.2]. The client walks `TransportPolicy.order` on repeated handshake failure; each rung has a failure budget (N handshakes / T seconds) before escalating [04_P2 §1.4].

```
// helix-core/src/ladder.rs  (sketch)
pub struct TransportPolicy {
    pub order: Vec<TransportKind>,   // pushed by coordinator;
        e.g. regional prior (§5.4)
    pub pin: Option<TransportKind>,  // user manual override —
        skip the ladder entirely
    pub budget: FailureBudget,       // { max_handshakes: u8,
        window: Duration }  per rung
}

// Default order (unrestricted network):
//   [ plain-udp ]
// Restricted prior (e.g. censored region, pushed by coordinator):
//   [ plain-udp, lwo, masque-h3, shadowsocks, udp-over-tcp ]
```

Selection algorithm [04_ARCH §3.2, 04_P2 §1.4]:

1. Start at `pin` if set (manual override), else `order[0]`.
2. `dial()` the rung; on `Connected` within budget → done; emit `Connected{transport,rtt}`.
3. On `DialTimeout`/`HandshakeFailed`/`EndpointBlocked` exceeding the rung's failure budget → escalate to the next rung; emit `Reconnecting`.
4. On success, **remember the working transport per-network** (SSID / gateway fingerprint) so reconnects on the same hostile network

skip straight to what worked — the difference between “VPN that sometimes connects” and “VPN that just works” [04_P2 §1.4].

5. The coordinator may push a **regional prior** (e.g. clients from CN-resolved IPs start at `shadowsocks`) via `TransportPolicy.order`, so users in censored regions do not pay the escalation latency every time [04_P2 §1.4].
6. Telemetry records **only** “transport X succeeded after N escalations in region R” (aggregate, no per-user data, I5) → feeds the Censorship-Evasion Success dashboard [04_ARCH §9, 04_P2 §1.4].

5.4 Sequence diagram — the transport escalation ladder

```
sequenceDiagram
    autonumber
    participant App as Client app (Flutter)
    participant Orch as Orchestrator (helix-core)
    participant Lad as Ladder (TransportPolicy)
    participant Tx as Transport (dial)
    participant Edge as Gateway edge (:443)

    App->>Orch: start(transport = auto)
    Orch->>Lad: select(order = [plain-udp, lwo, masque-h3,
shadowsocks, udp-over-tcp])
    Orch-->>App: status = Connecting

    Note over Lad,Edge: Rung 1 — plain-udp (default fast path)
    Lad->>Tx: dial(PlainUdp)
    Tx->>Edge: WG handshake over UDP :51820
    Edge--xTx: dropped (DPI blocks WG/UDP) → HandshakeFailed
    Tx-->>Lad: Err(HandshakeFailed), budget exceeded

    Note over Lad,Edge: Rung 2 — lwo (cheap header obfs)
    Orch-->>App: status = Reconnecting
    Lad->>Tx: dial(Lwo)
    Tx->>Edge: WG handshake over mangled UDP
    Edge--xTx: dropped (signature still caught) → HandshakeFailed
    Tx-->>Lad: Err(HandshakeFailed), budget exceeded

    Note over Lad,Edge: Rung 3 — masque-h3 (CONNECT-UDP / HTTP-3
on :443)
    Lad->>Tx: dial(MasqueH3)
    Tx->>Edge: QUIC/H3 CONNECT-UDP (RFC 9298) on :443/udp
    Edge-->>Tx: 200 (flow established), looks like HTTP/3
    Tx->>Edge: WG datagrams as HTTP Datagrams (RFC 9297/9221)
```

```
    Edge-->>Tx: WG handshake reply
    Tx-->>Lad: Ok(Connected, rtt = 23 ms)
    Lad->>Lad: remember per-network: this SSID → start at masque-h3 next time
    Orch-->>App: status = Connected { transport: "masque-h3", rtt_ms: 23 }

    Note over Orch,Edge: if masque-h3 also fails → escalate to shadowsocks → udp-over-tcp
```

---

6. Overlay addressing & multi-network routing

This is the part the original ops runbook never addressed [04_ARCH §3.4]. A single user reaches **N joined private networks**, which may have **colliding RFC1918 ranges** (two connectors each serving `192.168.1.0/24`) — that collision MUST be solved at v1 [SYNTHESIS §3 D4].

6.1 Overlay addressing — ULA /48 + 4via6 (decision D4, recommended)

> **DECISION D4 — IP-subnet collision across N joined networks. SURFACED** [SYNTHESIS §3 D4, 04_ARCH §3.4].
>
> - **Camp A (recommended, 04_ARCH): IPv6 ULA /48 per tenant** (`fd7a:helix:<tenant>::/48`)
>   - Tailscale-style **4via6** mapping of advertised IPv4 LANs, so overlapping `192.168.1.0/24`s across different connectors never collide.
> - **Camp B (GMI/KMI): CGNAT `100.64.0.0/10`, 1:1 per network** — assign each joined network a distinct slice of the CGNAT space and 1:1-NAT into it.
>
> **Recommendation:** adopt **Camp A (ULA /48 + 4via6)**. It scales to far more networks than a hand-partitioned CGNAT space, is the proven Tailscale model, and is IPv6-native (the future default). Surface 4via6 as the engine but **hide it behind Console UX** —

> 4via6 mapping is powerful but confusing to end users (a named open risk) [04_ARCH §13]. CGNAT 1:1 remains a documented fallback for IPv4-only edge environments.

- Every node (client, connector, advertised host) gets a **stable overlay address** in the tenant's ULA /48 [04_ARCH §3.4].
- An advertised IPv4 LAN `10.10.0.0/24` behind connector A is mapped to a deterministic IPv6 prefix via 4via6: `fd7a:helix:<tenant>:<siteID>:<4via6-encoded-v4>`. The `siteID` disambiguates two connectors that both serve `192.168.1.0/24`: each gets a distinct overlay prefix, and the gateway NATs into the correct connector [04_ARCH §3.4 overlapping-CIDR handling].

6.2 Prefix advertisement & routing map

Connectors advertise their served CIDRs to the control plane; the control plane compiles a **routing map** (which connector is next hop for which overlay prefix) and pushes it to the gateway edge and to authorized clients via `WatchNetworkMap` (doc 03) [04_ARCH §3.4]. The data plane consumes the map into `helix-route`:

```
// helix-route/src/map.rs  (the desired-state the reconciler
        converges to; doc 03 streams it)
pub struct RouteMap {
    pub self_overlay: IpAddr,                  // this node's
        overlay IP
    pub peers: Vec<PeerRoute>,                 // already policy-
        filtered (need-to-know, I6)
    pub dns: Vec<IpAddr>,
}
pub struct PeerRoute {
    pub wg_pubkey: [u8; 32],
    pub endpoint_candidates: Vec<SocketAddr>,  // for §8 NAT
        traversal
    pub allowed_ips:
        Vec<IpNet>,               // overlay prefixes this peer is
        the next hop for
    pub via_connector: Option<SiteId>,         // 4via6 site
        disambiguation
}
```

Phase-0 fakes the stream with a static `map.json` of the *exact shape* Phase 1's `WatchNetworkMap` streams, so the reconciler is real even though the source is static [04_P0 §10].

6.3 Reconciliation (push, don't poll)

Agents are **declarative reconcilers**: they diff desired-vs-actual `RouteMap` and converge — bring peers up/down, switch transport, update routes — **without restarting** [04_ARCH §4.4, 04_P0 §10]. Convergence target: a route/policy change reflected on all affected edges in **< 1 second** [04_ARCH §4.4]. The reconciler never polls; it reacts to map deltas pushed on the open stream (file-watch stands in during Phase 0).

---

7. Policy / ACL → AllowedIPs + nftables/eBPF verdict map

Default-**deny** (I6). A declarative allow-list (Tailscale-ACL-flavored, `group:contractors → net:warehouse-cameras:554/tcp`) is compiled by the control plane and expressed at the data plane as **per-peer `AllowedIPs` plus an nftables/eBPF verdict map** on the edge [04_ARCH §3.4, §7, SYNTHESIS §4].

Two enforcement layers — **both** required (defense in depth):

1. **WireGuard `AllowedIPs`** (cryptographic routing): each peer's `AllowedIPs` is the set of overlay prefixes that peer is allowed to send/receive — WG itself drops a packet whose source does not match the peer's allowed set. This is the coarse, crypto-enforced layer.
2. **Edge verdict map** (port/proto-granular): an **nftables verdict map** (Phase 1) or **eBPF** program (Phase 2, scale) on the gateway evaluates the compiled policy rule (`src_group → dst_selector : ports : action`) per flow. Default-deny; fail-**closed** (if the compiled policy is absent/stale, drop) [04_ARCH §7, SYNTHESIS §4].

```
// helix-route/src/policy.rs  (compiled verdict the edge installs)
pub struct CompiledPolicy { pub version: u64, pub rules:
        Vec<VerdictRule> }
pub struct VerdictRule {
    pub src: IpNet, pub dst: IpNet, pub proto: L4Proto, pub
        ports: PortRange, pub action: Verdict,
}
pub enum Verdict { Allow, Drop }   // default Drop (I6)
pub enum L4Proto { Any, Tcp, Udp, Icmp }

# edge-installed nftables verdict map (Phase 1 representation;
eBPF equivalent in Phase 2)
```

```
table inet helix {
    map policy_v {
        type ipv6_addr . ipv6_addr . inet_proto . inet_service :
verdict
        elements = {
            fd7a:helix:1::a . fd7a:helix:1:warehouse::20 . tcp .
554 : accept
        }
    }
    chain forward {
        type filter hook forward priority 0; policy drop;   #
DEFAULT-DENY (I6)
        ip6 saddr . ip6 daddr . meta l4proto . th dport vmap
@policy_v
    }
}
```

Properties [04_ARCH §3.4, §7]: **split-horizon** — connectors cannot reach each other unless policy says so; clients cannot reach networks they are not granted; microsegmentation is the default. Revocation: `device.revoked` (doc 02) → the verdict map drops the peer in **< 1 second** with no restart [04_ARCH §4.4, SYNTHESIS §7]. **Peers are delivered to each agent already policy-filtered** (need-to-know) so the client never even learns of networks it cannot reach (I6) [SYNTHESIS §7].

---

8. Direct peer-to-peer & NAT traversal (Phase 2 — additive)

Phase 1 relays **all** peer traffic through the gateway (simple, matches the proven slice). Phase 2 makes the gateway a **coordinator** + **relay-of-last-resort**, with traffic going **directly** between client and connector wherever NAT allows — lower latency, far less gateway bandwidth [04_P2 §3]. This is purely additive: it fills `PeerRoute.endpoint_candidates` (the field §6.2 reserves) and changes which endpoint WG latches onto — no transport or trait change.

1. **Endpoint discovery (STUN-like)** [04_P2 §3.1]: each node learns local candidates (all interface addresses) + server-reflexive candidates (probe the gateway's STUN-like endpoint, which echoes the observed `src ip:port`, revealing the NAT mapping). Candidates

are reported to the coordinator and distributed to authorized peers as `endpoint_candidates`.
2. **Hole punching** [04_P2 §3.2]: both peers simultaneously send WG handshake probes to all of the peer's candidates; for full-cone / restricted-cone / port-restricted NATs a path opens, and **WireGuard's roaming latches onto whatever source address a valid handshake arrives from** — no extra machinery. Symmetric-NAT-on-both-ends → hole punching fails → fall back to relay.
3. **Signaling = the existing stream** [04_P2 §3.3]: `WatchNetworkMap` *is* the signaling channel (the coordinator pushes each peer the other's candidate list as a delta) — no separate signal service (HelixVPN folds NetBird-style signaling into the coordinator).
4. **Relay fallback (DERP-style)** [SYNTHESIS §4]: when both ends are behind symmetric NAT, traffic relays through a `helix-relay` (gateway) — relay-of-last-resort, not the default.

---

9. DAITA — traffic-shaping stage (Phase 2, data-plane placement only)

DAITA defeats size/timing/frequency fingerprinting even of fully-encrypted flows. **Do not roll your own** — adopt **maybenot** (the framework behind Mullvad's DAITA): a state-machine engine for padding + cover-traffic "machines" [04_P2 §2, SYNTHESIS §4]. The data-plane spec owns only its **placement** and the per-packet hook; the privacy design (machines, off-by-default policy, website-fingerprinting validation harness) lives in the security doc.

Placement (L2.5): a stage **above WireGuard, below the transport** — it operates on the stream of WG datagrams (padding/injecting cover) before they hit the obfuscating transport, orthogonal to *which* transport runs [04_P2 §2.2].

```
TUN → WG encrypt → [DAITA: pad + inject cover] → Transport → wire
```

```rust
// helix-daita/src/lib.rs  (sketch)
pub struct Daita { framework: maybenot::Framework /* machines are
        CONFIG from NetworkMap, not code */ }
impl Daita {
    /// Per outgoing WG datagram: may pad it and/or schedule cover
        packets.
    pub fn on_packet(&mut self, dg: &mut Bytes, now: Instant) ->
        Vec<ScheduledAction> { /* … */ }
    /// Timer-driven: emit scheduled cover/padding packets.
```

```
    pub fn tick(&mut self, now: Instant) -> Vec<Bytes> { /* … */ }
}
```

Machines are distributed as **data** via a `daita` field on the `NetworkMap`, so defenses tune server-side without a client rebuild [04_P2 §2.3]. Off by default, opt-in ("maximum privacy" with an honest bandwidth/latency cost note) [04_P2 §2.3].

---

10. MTU & overhead budget

The transport changes how many bytes of WG payload fit per L1 packet; each impl reports its `effective_mtu()` and the orchestrator sets the inner WG MTU accordingly [04_P0 §4.2, §5.2].

Transport	`effective_mtu()`	Rationale	Source
`plain-udp`	**1420**	standard WG-over-IPv4 (1500 − 20 IP − 8 UDP − 32 WG hdr/tag ≈ 1420)	[04_P0 §4.3]
`masque-h3`	**1280** (measure & tune)	QUIC + HTTP-Datagram + UDP-proxy overhead eats headroom; 1280 is IPv6 min-MTU floor	[04_P0 §5.2]
`connect-ip`	~1300	IP-over-H3, no inner-WG header but QUIC overhead	derived
`shadowsocks`	~1380	TCP MSS − 2-byte length prefix − AEAD tag	derived [04_P2 §1.1]
`udp-over-tcp`	~1380	TCP MSS − length prefix	derived [04_P2 §1.2]
`lwo`	~1400	plain-udp minus padding bytes	derived [04_P2 §1.3]

Rules: (1) the inner WG MTU = `min(active transport.effective_mtu(), path-MTU-discovered)`; (2) when DAITA pads, padding is added *after* WG encrypt and counts against the transport budget, not the inner MTU; (3) `Oversize` is a hard error (`TransportError::Oversize`) — the orchestrator must fragment at L3 (rare) or lower the inner MTU rather than silently truncate. The MASQUE MTU penalty is **quantified in Phase 0** (record MTU/throughput/CPU vs plain-udp) so the ladder's cost model is real [04_P0 §5.3].

---

11. Multi-hop (nested WireGuard) + edge-language & topology decisions

11.1 Multi-hop — nested WG with per-hop keys

Generalize the original "chain two VPSes" [04_ARCH §3.5]. A client may route `Client → Gateway-Entry → Gateway-Exit → {internet | connector}`. **Entry sees the client but not the destination; exit sees the destination but not the client** — the Mullvad multi-hop property. Implemented as **nested WireGuard with per-hop keys**: the client holds a WG session to Entry *and* a WG session to Exit; the Exit session's datagrams are themselves the payload of the Entry session. Orchestrated by the control plane and pushed as a multi-hop `RouteMap` (`PeerRoute` chain) [04_ARCH §3.5]. The transport layer is unchanged — each hop is an ordinary `Transport` carrying that hop's WG datagrams (the outer hop may be `masque-h3`, the inner `plain-udp`).

```
// helix-core: a hop chain is just an ordered Vec of WgPeer +
        Transport pairs
pub struct HopChain { pub hops: Vec<Hop> }            // [Entry,
        Exit] for 2-hop
pub struct Hop { pub wg: WgPeer, pub transport: Box<dyn
        Transport> }
// outbound: encrypt for Exit (innermost) → encrypt for Entry
        (outermost) → Entry transport.send()
```

11.2 Decision D5 — gateway edge language (MASQUE termination)

> **DECISION D5 — gateway edge language. SETTLED BY PHASE-0 G4 BENCHMARK** [SYNTHESIS §3 D5, 04_P0 §7].
>
> - **Camp A (recommended, 04_P0): Rust** (`quinn` + `h3` + hand-rolled CONNECT-UDP) — the edge shares `helix-transport` **byte-for-byte** with clients (I4); no GC on the hot path.
> - **Camp B: Go** (`quic-go` + `masque-go`, turnkey CONNECT-UDP/IP) — matches the Go/Gin control plane, more mature MASQUE tooling; cost = dual MASQUE implementations to keep in sync.
>
> **Recommendation / decision rule** [04_P0 §7.3]: choose **Rust** if it gets within ~10–15% on throughput/CPU-per-Gbps and the hand-rolled CONNECT-UDP is not a quagmire (the single-implementation win). Choose **Go** if MASQUE-in-Rust proves painful or Go wins decisively on cost-to-serve, and accept dual implementations mitigated by **shared test vectors + a conformance suite across both edges**. The call is recorded in the Phase-0 decision log with benchmark numbers (throughput @ 1/10/100 clients, CPU/Gbps, p99 latency incl. GC tail, handshake churn/sec, memory under churn) [04_P0 §7.2, §8]. **This spec assumes Rust** and notes the Go path as the conformance-suite-mitigated fallback.

11.3 Decision D6 — transport topology (per-leg)

> **DECISION D6 — transport topology. SURFACED** [SYNTHESIS §3 D6, 11_MST].
>
> - **Default (04_ARCH):** a single protocol family end-to-end (WG everywhere, the obfuscating transport chosen per the ladder).
> - **Camp B (11_MST, distinctive): asymmetric per-leg** — Hysteria2/QUIC on the *user ↔ gateway* leg (where obfuscation matters most), plain WireGuard on the *gateway ↔ networks* leg (where the connector dials out of a cooperative network).
>
> **Recommendation:** the `Transport` trait already makes per-leg topology free — each leg is an independent `Transport` selection. Adopt **best-fit-per-leg as a policy, not a new mechanism**: the coordinator pushes a `TransportPolicy.order` per leg, so the user ↔ gateway leg can start at `masque-h3`/`hysteria2` while the

> gateway ↔ connector leg stays `plain-udp` on cooperative networks. No code change beyond per-leg `TransportPolicy` — surface it as the default Phase-2 behavior.

---

12. File-by-file build checklist (data plane)

Phase-tagged; each item maps to a workable item in the §11.4.93/.95 SQLite tracker [SYNTHESIS §9].

Crate / file	Owns	Phase	Phase-0 gate
`helix-transport/src/lib.rs`	`Transport` trait, `TransportConfig`, `dial()`, registry	0	S0
`helix-transport/src/plain_udp.rs`	baseline UDP carrier	0	G1 / S1
`helix-transport/src/masque.rs`	CONNECT-UDP / HTTP-3 / QUIC datagrams	0	G2 / S3
`helix-transport/src/error.rs`	`TransportError` taxonomy	0	S0
`helix-wg/src/lib.rs`	boringtun `Tunn` wrapper, `WgVerdict`	0	G1 / S1
`helix-tun/src/lib.rs`	Linux TUN; fd-injection for shims	0	S1
`helix-core/src/orchestrator.rs`	three loops, client/connector posture	0	S2
`helix-core/src/status.rs`	`TunnelStatus` broadcast enum	0	G5 / S5
`helix-core/src/ladder.rs`	`TransportPolicy`, auto-selection	1	—
`helix-route/src/map.rs`	`RouteMap` reconciler (push, no restart)	0→1	G6 / S8
`helix-route/src/policy.rs`	ACL→`AllowedIPs`+verdict-map compiler	1	—
	SS-wrap carrier	2	—

Crate / file	Owns	Phase	Phase-0 gate
`helix-transport/src/shadowsocks.rs`			
`helix-transport/src/udp_over_tcp.rs`	UoT last-resort	2	—
`helix-transport/src/lwo.rs`	lightweight obfs (basic P1, hardened P2)	1→2	—
`helix-transport/src/connect_ip.rs`	RFC 9484 IP-over-H3 (feature-gated)	2	—
`helix-transport/src/hysteria2.rs`	Hysteria2+Salamander carrier (feature-gated, D1)	1→2	—
`helix-daita/src/lib.rs`	maybenot shaping stage	2	—
`helix-core/src/hop_chain.rs`	nested-WG multi-hop	2	—
`bin/helix-edge.rs`	gateway MASQUE termination (D5)	0	G4 / S4

12.1 Anti-bluff acceptance evidence (constitution §11.4.5/.69/.107, SYNTHESIS §9)

Every data-plane claim ships captured evidence, not config-only PASS:

- **G1 plain-UDP slice** — `iperf3` through-tunnel ≥ 80% bare link + `ping` to LAN host succeeds; CSV from `bench.sh` [04_P0 §0, §8].
- **G2 MASQUE through DPI block** — slice works with plain WG **blocked** (`nft` rule), `tshark` capture classifies the flow as HTTP/3 with **no WG signature**, goodput @ 5% `netem` loss beats the UoT strawman [04_P0 §5.3, §8].
- **G6 reconcile** — edit the static map → peer reachable, **no restart** of unrelated state [04_P0 §10].
- Per-transport: `effective_mtu()` measured (not assumed); ladder escalation captured as an ordered `TunnelStatus` event trace; verdict-map default-deny proven by a denied-flow capture.

13. What is explicitly out of scope for this document

Control-plane services, the `WatchNetworkMap` protobuf/Connect contract, Postgres data model & RLS, enrollment/PKI, event bus (doc 02/03); client UI, design system, FFI Dart bindings (doc 05); platform tunnel shims beyond the TUN-fd handoff contract — iOS NEPacketTunnelProvider memory ceiling (G3, the make-or-break gate), Android VpnService/JNI, Windows wireguard-nt, HarmonyOS Network Kit, Aurora Qt/tun (doc 06); deployment quadlets, observability, HA (doc 07/08). The post-quantum PSK, DAITA privacy policy, and kill-switch/DNS-leak state machine are *referenced* here for their data-plane seams and *specified* in the security doc.

End of data-plane specification (document 01). Pair with doc 03 (`WatchNetworkMap`* wire contract — the live source of the static* `map.json` *seam at §6.2) and doc 06 (platform tunnel shims — the TUN-fd handoff at §5.1). Surviving interfaces from Phase 0 (the* `Transport` *trait §3.1,* `helix-wg` *§4, the* `TunnelStatus` *enum §5.2, the* `RouteMap` *§6.2) are frozen contracts; their implementations may evolve [04_P0 §14].*